

Fault Tolerance

- Fault Tolerance
- Recovery

Fault Tolerance

Introduction

What are the various faults that a distributed systems may face?

- a) Failure of a link
- b) Failure of a site
- c) Loss of message
- d) Failure of power

What steps are required for recovery from failure?

- a) Post repairing integration with main system should happen smoothly and gracefully
- b) Upon link failure both parties at end must be notified
- c) Mechanisms for recovery from faults must be adopted
- d) Failures must be logged systematically

System Failure Modes

- **Failures unique to distributed systems:**
 - **Failure of a site.**
 - **Loss of messages**
 - Handled by network transmission control protocols such as TCP-IP
 - **Failure of a communication link**
 - Handled by network protocols, by routing messages via alternative links
 - **Network partition**
 - A network is said to be partitioned when it has been split into two or more subsystems that lack any connection between them
 - Note: a subsystem may consist of a single node
- **Network partitioning and site failures are generally indistinguishable.**

Which of the following approaches are used to achieve reliable systems?

- a) Fault prevention
- b) Fault removal
- c) Fault tolerance
- d) All of the mentioned

Fault Tolerance

- A DS should be fault-tolerant
 - Should be able to continue functioning in the presence of faults
- Fault tolerance is related to **dependability**

Dependability

Dependability Includes

- Availability
- Reliability
- Safety
- Maintainability

Availability & Reliability (1)

- **Availability:** A measurement of whether a system is *ready to be used immediately*
 - System is up and running at any given moment
- **Reliability:** A measurement of whether a system can *run continuously without failure*
 - System continues to function for a long period of time

Availability & Reliability (2)

- A system that never crashes but is shut down for a week once every year is 100% reliable but only 98% available
- A system goes down 1millisec/hr has an availability of more than 99.99%, but is unreliable

Safety & Maintainability

- **Safety:** A measurement of *how safe failures are*
 - System fails, nothing serious happens
 - For instance, high degree of safety is required for systems controlling nuclear power plants
 - **Maintainability:** A measurement of *how easy it is to repair a system*
 - A highly maintainable system may also show a high degree of availability
 - Failures can be detected and repaired automatically
- Self-healing systems

Faults

- A system **fails** when it cannot meet its promises (specifications)
- An **error** is part of a system state that may lead to a failure
- A **fault** is the cause of the error
- **Fault-Tolerance**: the system can provide services even in the presence of faults
- Faults can be:
 - **Transient** (appear once and disappear)
 - **Intermittent** (appear-disappear-reappear behavior)
 - A loose contact on a connector □ intermittent fault
 - **Permanent** (appear and persist until repaired)

Failure Models

Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	A server fails to respond to incoming requests A server fails to receive incoming messages A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure <i>Value failure</i> <i>State transition failure</i>	The server's response is incorrect The value of the response is wrong The server deviates from the correct flow of control
Arbitrary failure (Byzantine failure)	A server may produce arbitrary responses at arbitrary times

Consider two communication services for use in asynchronous distributed Systems. In service A, messages may be lost duplicated or delayed and Checksums apply only to headers. In service B, messages may be lost, delayed or delivered too fast for the recipient to handle them but those that are delivered arrive ordered and with the correct contents

Describe the classes of failure exhibited by each service
Classify their failures according to their effect on properties of validity and integrity

Can service B be described as a reliable communication service?

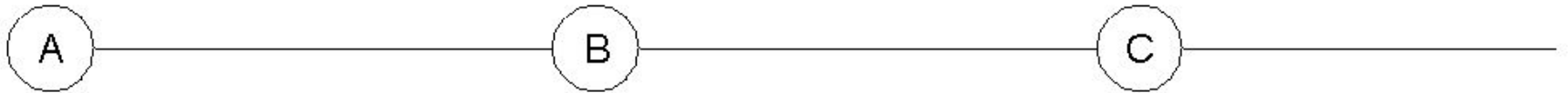
- Consider a simple server that carries out clients requests without accessing other servers. Why it is generally not possible to set a limit on the time taken by such a server to respond to a client request? What would need be done to make the server able to execute requests within a bounded time?

- We cannot send a limit for the time taken by a server to respond a client request because the arrival of client requests is not predictable. For example, we can have limits for executing a computation corresponding to a request but we can not predict how much a request will have to wait until its execution time arrives

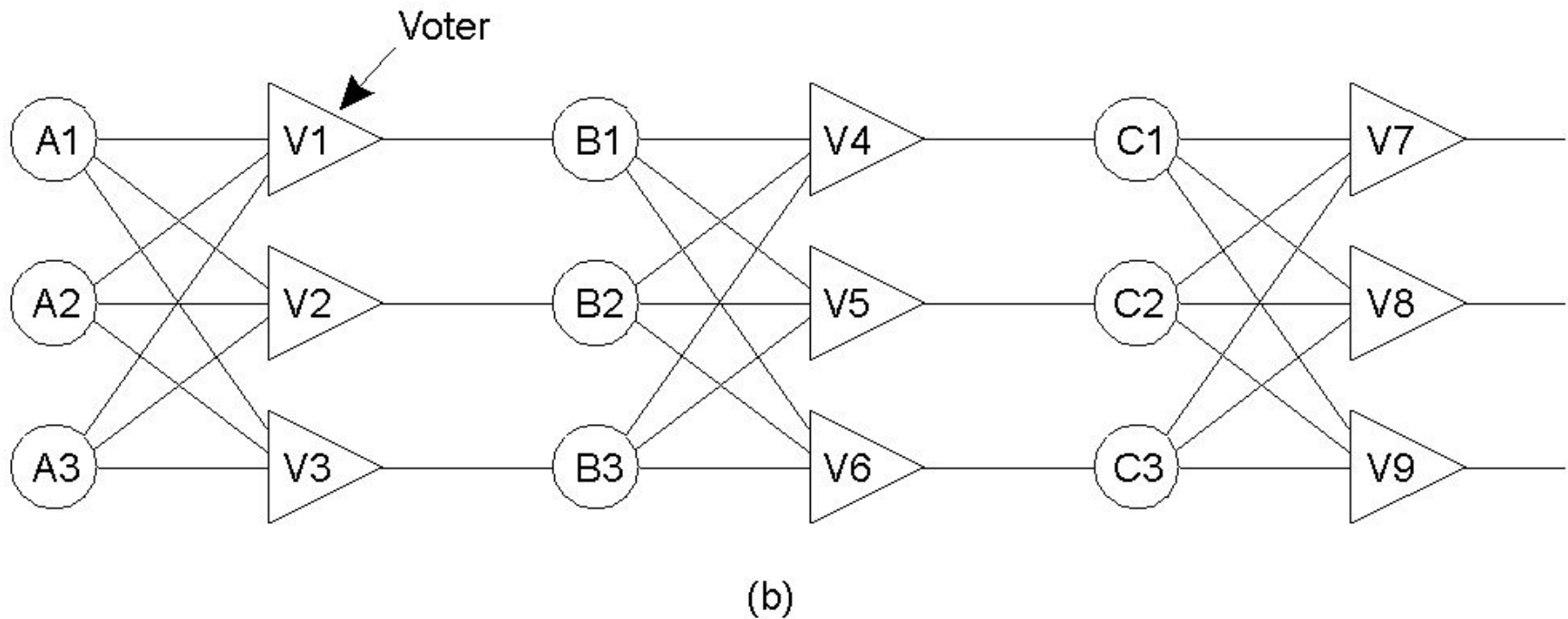
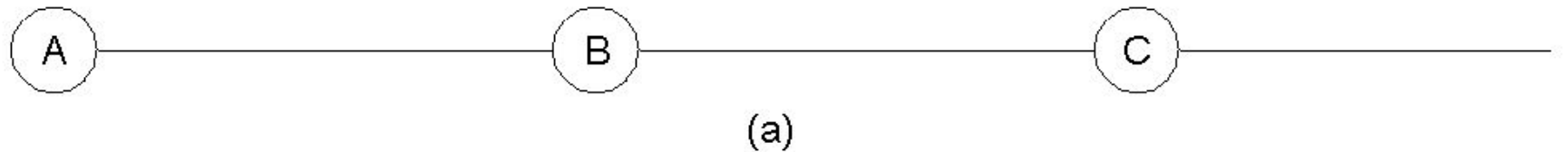
Failure Masking

- **Redundancy** is key technique for hiding failures
- Redundancy types:
 1. Information: add extra (control) information
 - Error-correction codes in messages
 2. Time: perform an action persistently until it succeeds:
 - Transactions
 3. Physical: add extra components (S/W & H/W)
 - Process replication, electronic circuits

Example – Redundancy in Circuits (1)



Example – Redundancy in Circuits (2)



Triple modular redundancy.

- A Web browser returns an outdated cached page instead of a more recent one that had been updated at the server.

Is this a failure, and if so, what kind of failure?

- Whether or not it is a failure depends on the consistency that was promised to the user.
- If the browser promises to provide pages that are at most T time units old, it may exhibit performance failures. However, a browser can never live up to such a promise in the Internet.
- A weaker form of consistency is to provide one of the client-centric models. In that case, simply returning a page from the cache without checking its consistency may lead to a response failure.

Fault Tolerance

Process Resilience

Process Resilience

- Mask process failures by replication
- Organize identical processes into groups, a message sent to a group is delivered to all members
- If a member fails, another can take over it

Process Replication for Failure Masking

A system is k fault-tolerant if it can survive and function even if it has k faulty processes

- For crash failures (a faulty process halts, but is working correctly until it halts) replicas required: $k+1$
- For Byzantine failures (a faulty process may produce arbitrary responses at arbitrary times) replicas : $2k+1$

Process Groups

Examples of some group applications

- **Replicated file systems:** All file servers constitute a group. Files are replicated at every fileserver to enhance file availability and reliability
- **Replicated program executions** for the resiliency of computations.

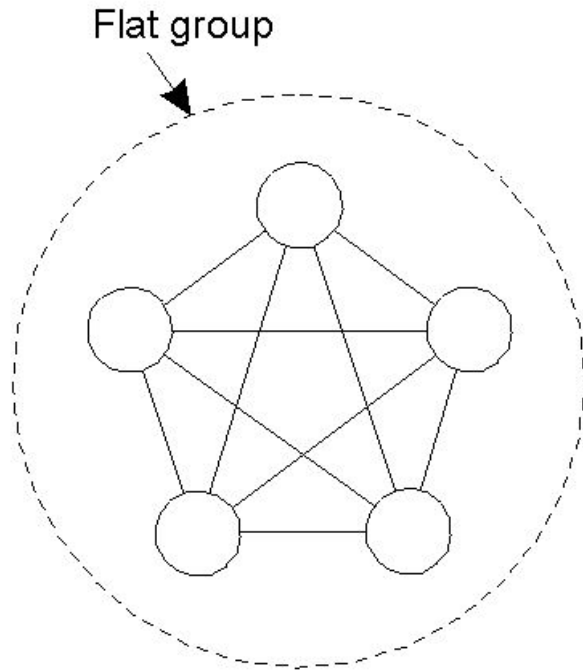
Process Groups

- **A distributed database** model has a transaction manager (TM) and a data manager (DM) on each site.

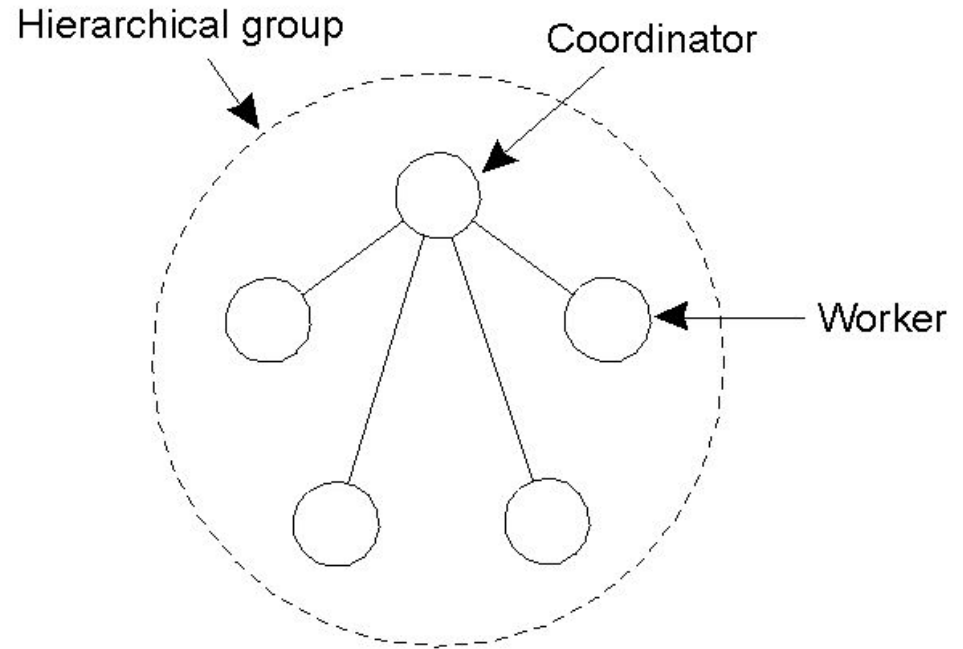
Each TM accepts user requests and translates them into commands for DMs. Each DM maintains part of the database stored at its site and may concurrently execute transactions from multiple TMs.

A **transaction group** consists of all DMs participating in the same transaction.

Flat Groups versus Hierarchical Groups



(a)



(b)

- a) Communication in a flat group.
- b) Communication in a simple hierarchical group

Fault Tolerance

Reliable Group Communication

Reliable Group Communication

- **Reliable multicast**: message sent to a group must be delivered to each member of that group.
 - What if a process joins during communication?
 - What if a sending process crashes during communication?

Hence...

Distinction should be made between reliable communication in the presence of faulty processes and when operating correctly.

Ordered delivery required sometimes even in **multicasting**.

Small Scale Group

- For small scale group:
 - Reliable, connection oriented, point-to-point connections are feasible
 - Messages have sequence numbers
 - Messages received in order
 - Receivers acknowledge messages
 - Request retransmission of missing messages
 - Sender keeps message in buffer till acknowledgement received

Reliable One-Many Communication

- Reliable multicast

- Lost messages => need to retransmit

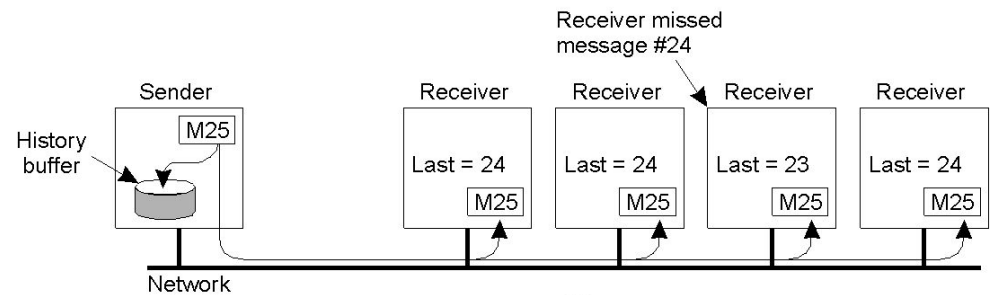
- Possibilities

- ACK-based schemes

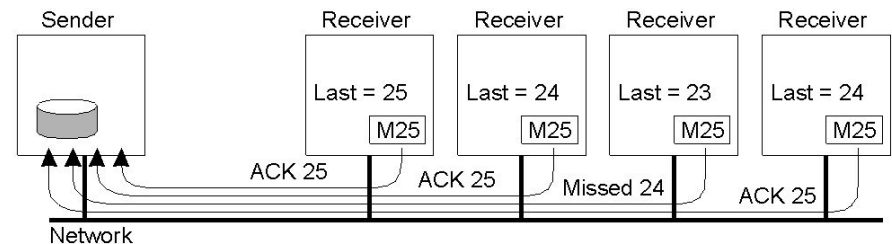
- Sender can become bottleneck- **feedback implosion**

- NACK-based schemes

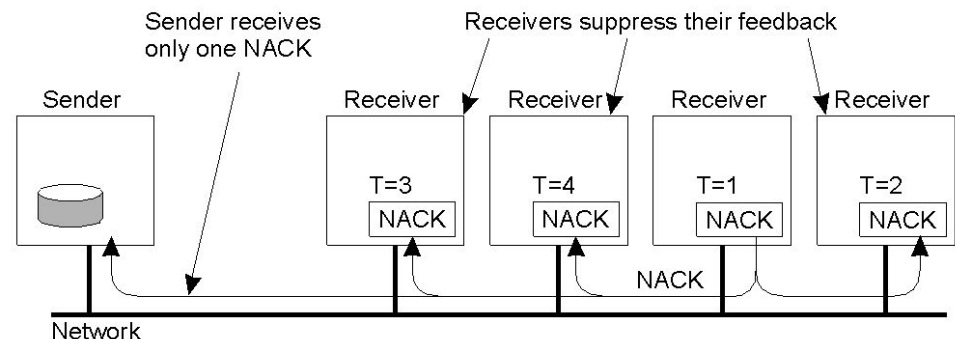
- Feedback suppression



(a)



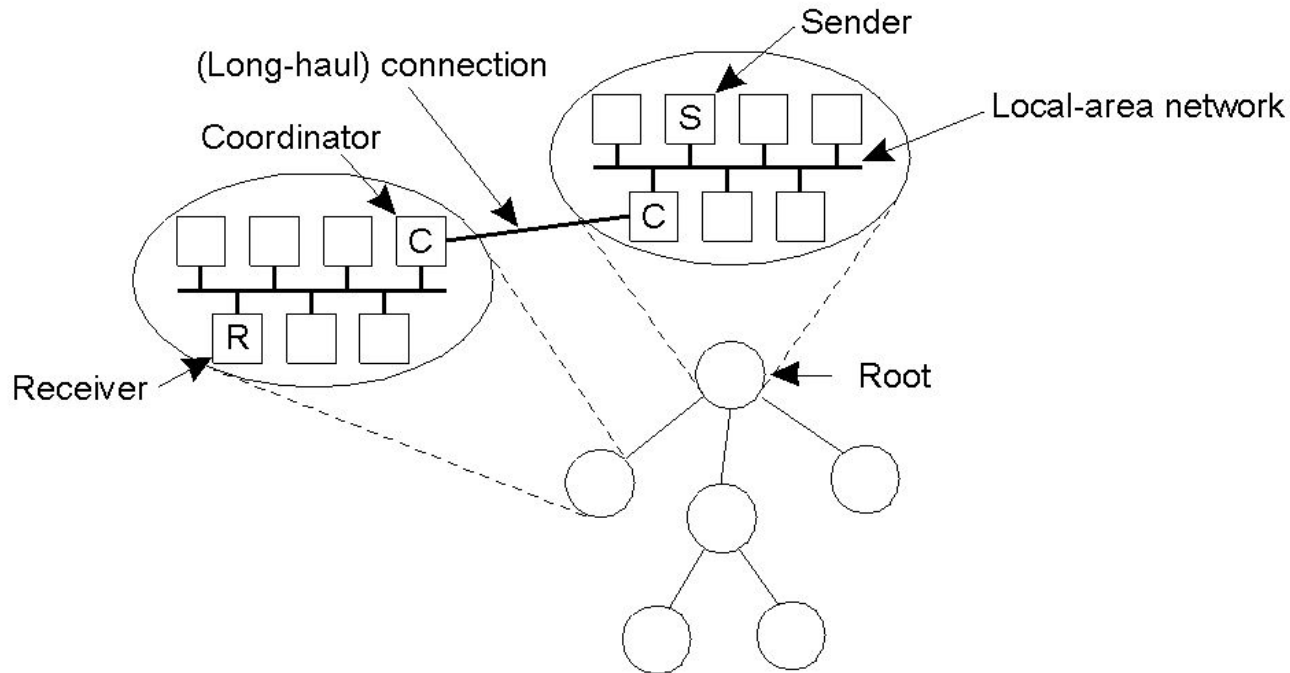
(b)



NACK based approach

- Receiver sends feedback only for missing message
 - Issues:
 - Sender is not aware of message having been delivered to entire group
 - Sender forced to keep transmitted messages in buffer “forever” to resolve retransmission requests

Hierarchical Feedback Control



The essence of hierarchical reliable multicasting (best for large process groups.

- a) Each local coordinator forwards the message to its children.
- b) A local coordinator handles retransmission requests

Atomic multicast:

RELIABLE MULTICASTING IN PRESENCE OF FAILURES:

When a message is sent to the group, all members of the group receive the message. Otherwise, no member receives the message. i.e., there is never the case that some members receive the message while others do not. It is all (member) or none.

Atomic multicast:

Example in which atomicity is needed:

- Replicated database is constructed as a group of processes, one process for each replica.
- Replicated data update.
- If not, the replicated data may get out of steps with each other.

Example in which atomicity is not needed:

- Locating objects in distributed service. It is sufficient that the server holding the object receives the message. If the message to this server is lost, the client can try again (say on time-out).
- Multiple notification in a flight information display system

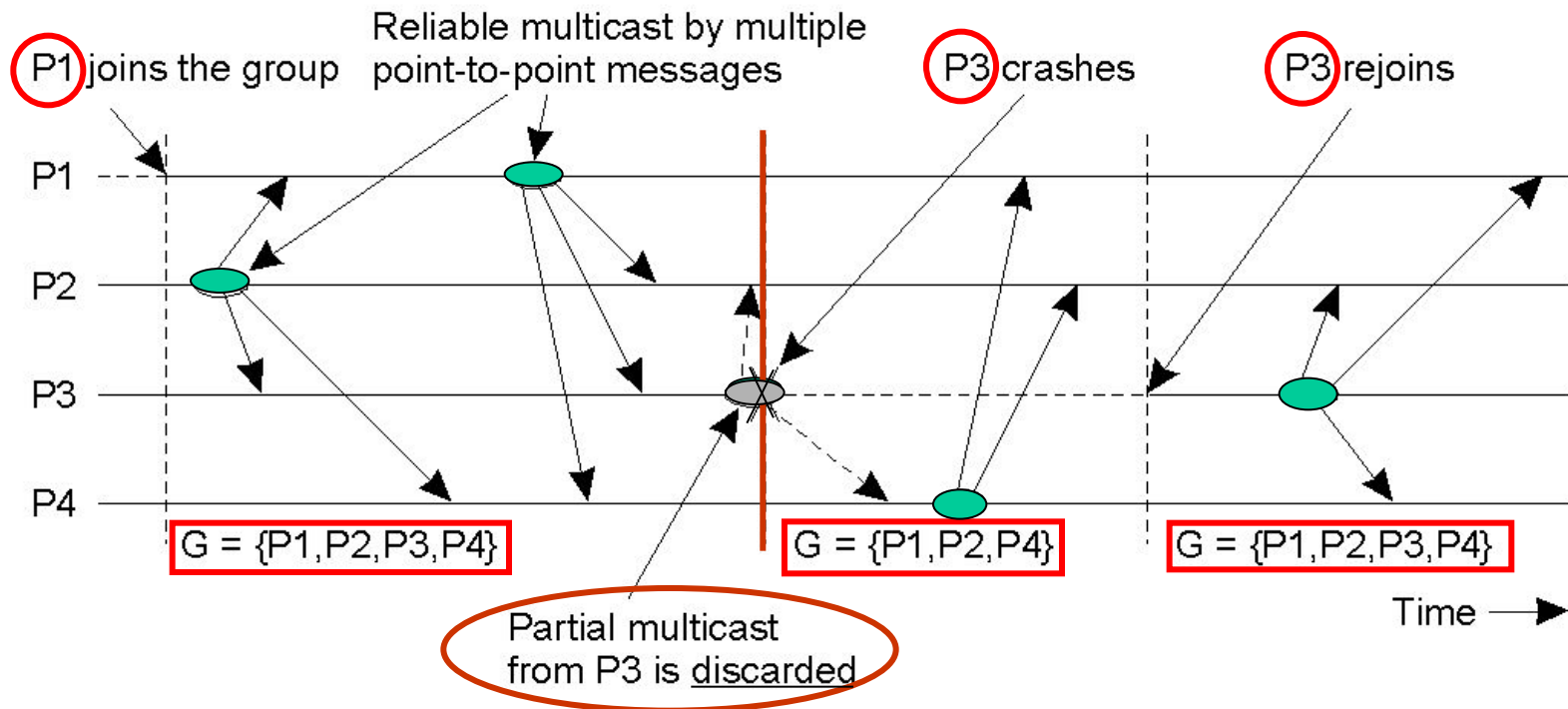
Atomic Multicast

- Atomic multicast: a guarantee that all process received the message or none at all
- Problem: how to handle process crashes?
- Solution: *group view*
 - Each message is uniquely associated with a group of processes
 - View of the process group when message was sent
 - All processes in the group should have the same view (and agree on it)

Virtual Synchrony

Virtual Synchrony guarantees that a message sent to a group view is delivered to each non-faulty member of the group.

If the sender crashes, the message may be either delivered to all the other processes or ignored by each of them.



- The principle of virtual synchronous multicast (view change similar to synchronization variable)

Message Ordering

Four different type of *ordering* of multicasts:

- Reliable, unordered multicast
- FIFO ordered multicast
- Causally ordered multicast
- Totally-ordered multicast

Virtually synchronous reliable multicasting offering **totally ordered** delivery is called **atomic multicasting**

Message Ordering

Process P1	Process P2	Process P3
sends m1	receives m1	receives m2
sends m2	receives m2	receives m1

- **Unordered multicast**
- Three communicating processes in the same group. The ordering of events per process is shown along the vertical axis.

Process P1	Process P2	Process P3	Process P4
sends m1	receives m1	receives m3	sends m3
sends m2	receives m3	receives m1	sends m4
	receives m2	receives m2	
	receives m4	receives m4	

- Four processes in the same group with two different senders, and a possible delivery order of messages under **FIFO-ordered multicasting**

--P1: m0, m1, m2

– P2: m3, m4, m5

– P3: m6, m7, m8

- FIFO? (m0, m3, m6, m1, m4, m7, m2, m5, m8)

- FIFO? (m0, m4, m6, m1, m3, m7, m2, m5, m8)

- P1: m0, m1, m2
- P2: m3, m4, m5
- P3: m6, m7, m8
- Cross-process happened-before: $m0 \sqsubset m4, m5 \sqsubset m8$

• Causal? (m0, m3, m6, m1, m4, m7, m2, m5, m8)

• Causal? (m0, m4, m1, m7, m3, m6, m2, m5, m8)

– P1: m0, m1, m2

– P2: m3, m4, m5

– P3: m6, m7, m8

• Total?

– P1: m7, m1, m2, m4, m5, m3, m6, m0, m8

– P2: m7, m1, m2, m4, m5, m3, m6, m0, m8

– P3: m7, m1, m2, m4, m5, m3, m6, m0, m8

• Total?

– P1: m7, m1, m2, m4, m5, m3, m6, m0, m8

– P2: m7, m2, m1, m4, m5, m3, m6, m0, m8

– P3: m7, m1, m2, m4, m5, m3, m6, m8, m0

Does FIFO ordered multicast imply causally ordered multicast?

Does total ordered multicast imply FIFO ordered multicast?

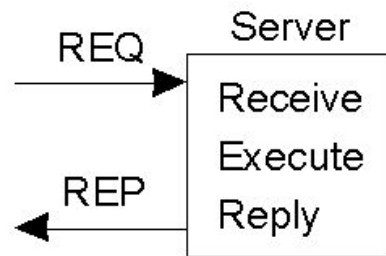
Fault Tolerance

Reliable Client-Server Communication

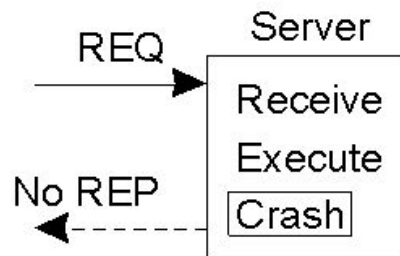
Reliable One-One Communication

- Earlier only Process failure considered,
- Need to consider **communication failures** too.
- Focus is **on masking crashes and omission failures**.
- Use reliable transport protocols (TCP) or handle at the application layer
- **TCP** masks omission failures (lost messages etc.) by ACKs and Retransmissions
- Crash Failure: when connection is abruptly broken (no more msgs can be transmitted through the channel)
- Client informed by raising exceptions
- Only way to mask: set up a new connection

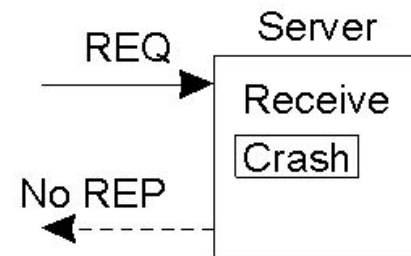
- RPC semantics in the presence of failures (make RPCs look like local ones), otherwise..
- Possibilities
 - Client unable to locate server
 - Lost request messages
 - Server crashes after receiving request
 - Lost reply messages
 - Client crashes after sending request



(a)



(b)



(c)

For each of the following applications, at-least once semantics or at-most once semantics is better?

- Reading and writing files from a server
- Compiling a program
- Remote banking

In a system where the client communicates with the server over an RPC, the client keeps sending a request (say operation $x()$) to the server until the server responds with the result of the request.

What RPC failure semantics is being implemented in this case?

How may the client and server be implemented to achieve this semantics

Fault Tolerance

Recovery

Recovery

- What happens after a fault has occurred ?
- A process that exhibits a failure has to be able to recover to a correct state
- There are two basic types of recovery:
 - Backward Recovery
 - Forward Recovery

Backward Recovery

- The goal of *backward recovery* is to bring the system from an erroneous state back to a prior correct state
- The state of the system must be recorded - *checkpointed* - from time to time, and then restored when things go wrong
- Examples
 - Reliable communication through packet retransmission

Forward Recovery

- The goal of *forward recovery* is to bring a system from an erroneous state to a correct new state (not a previous state)
- Examples:
 - When a lost/damaged packet can be reconstructed as a result of the receipt of other successfully delivered packets, then this is known as *Erasure Correction*. This is an example of a forward recovery technique

More on Backward Recovery

- Backward recovery is far more widely applied
- The goal of *backward recovery* is to bring the system from an erroneous state back to a prior correct state
- But, how to get a prior correct state?
 - Checkpointing:
 - Periodically checkpoint state
 - Upon a crash roll back to a previous checkpoint with a *consistent state*

Checkpointing

- Related to checkpointing are the *global state* and the *distributed snapshot algorithm*

Determining Global States

- **The global state of a distributed computation is**
 - *the set of local states of all individual **processes** involved in the computation*
 - +
 - *the states of the **communication channels***
- **How?**

Global State

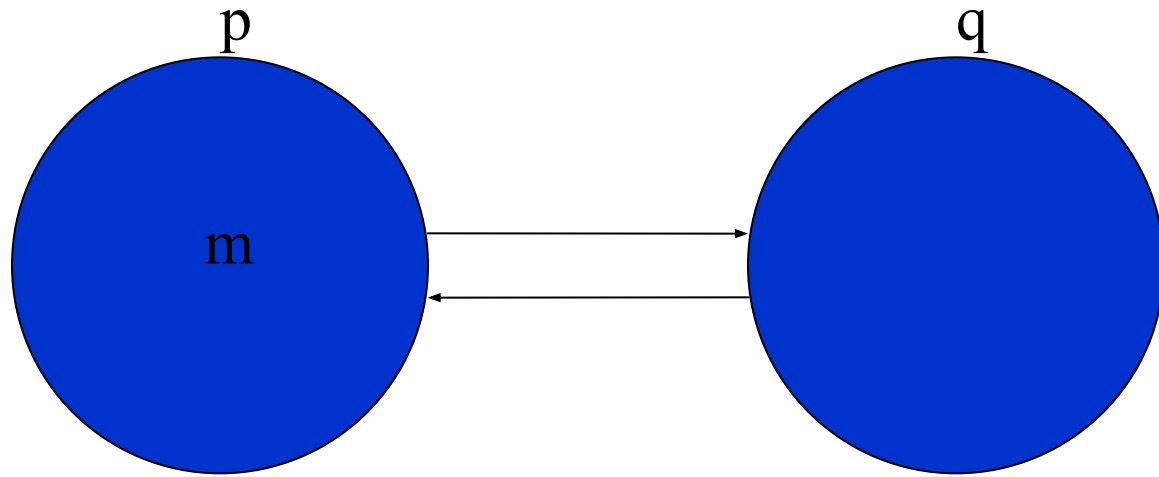
- We cannot determine the exact global state of the system, but we can record a snapshot of it

A naïve snapshot algorithm

- Processes record their states at *any* arbitrary points
- A designated process collects these states
- + So simple!!
- - Correct??

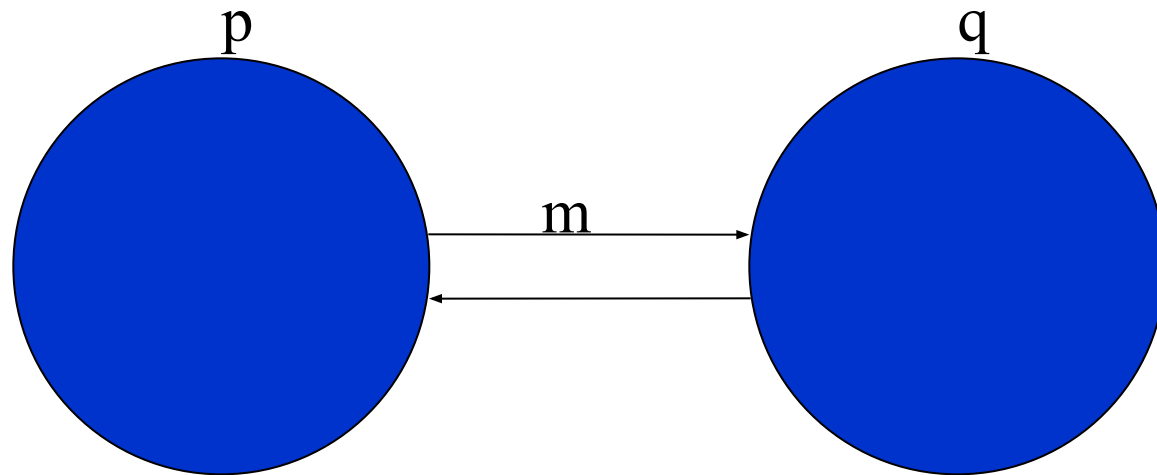
Example

Producer Consumer problem

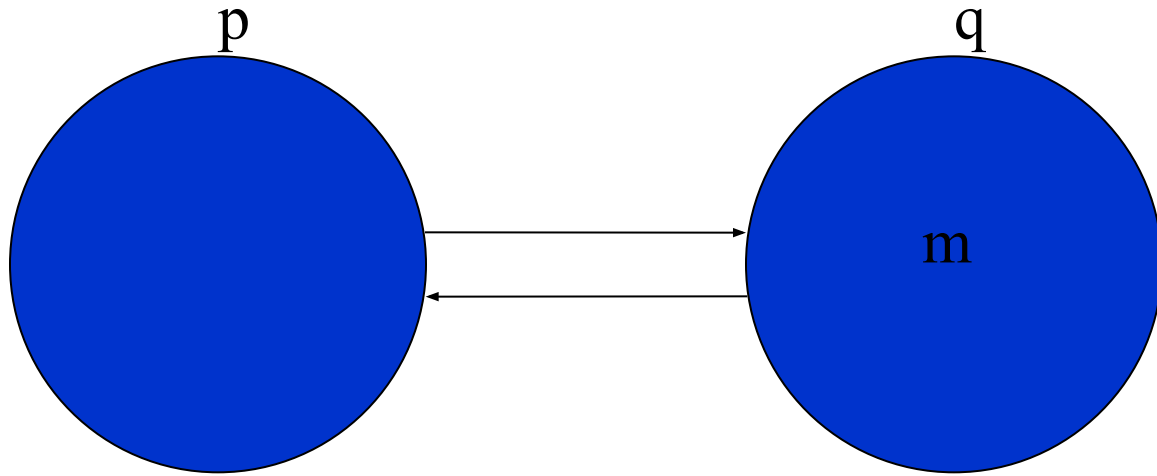


- p records its state

Example



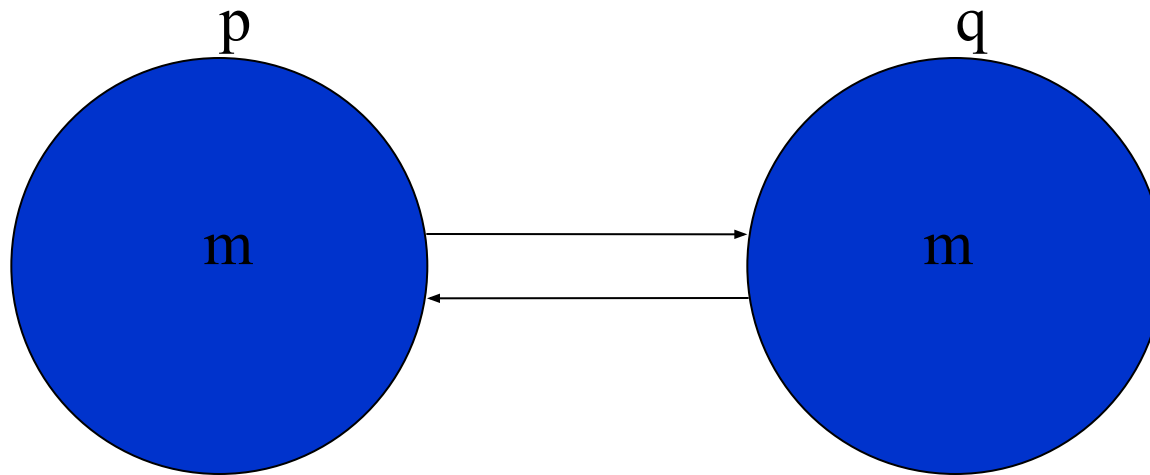
Example



- q records its state

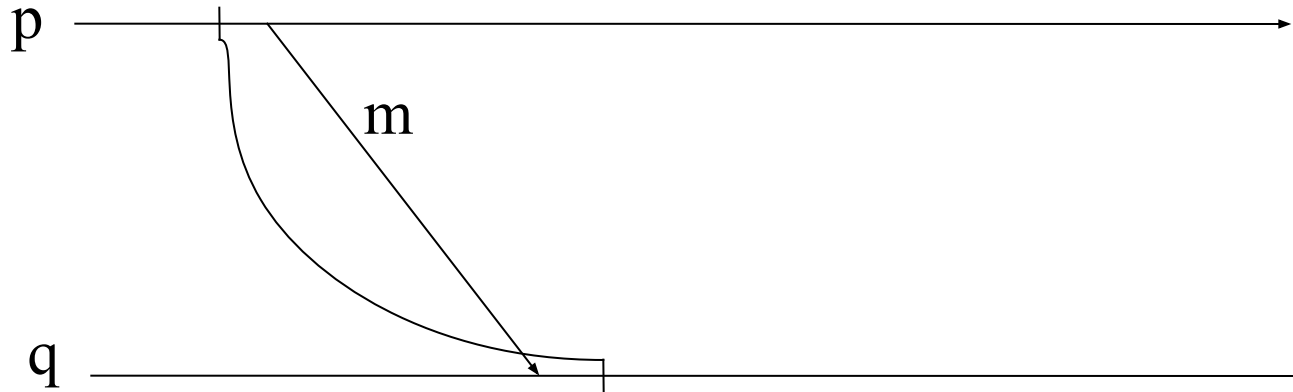
Example

The recorded state



The sender has *no* record of the sending
The receiver *has* the record of the receipt

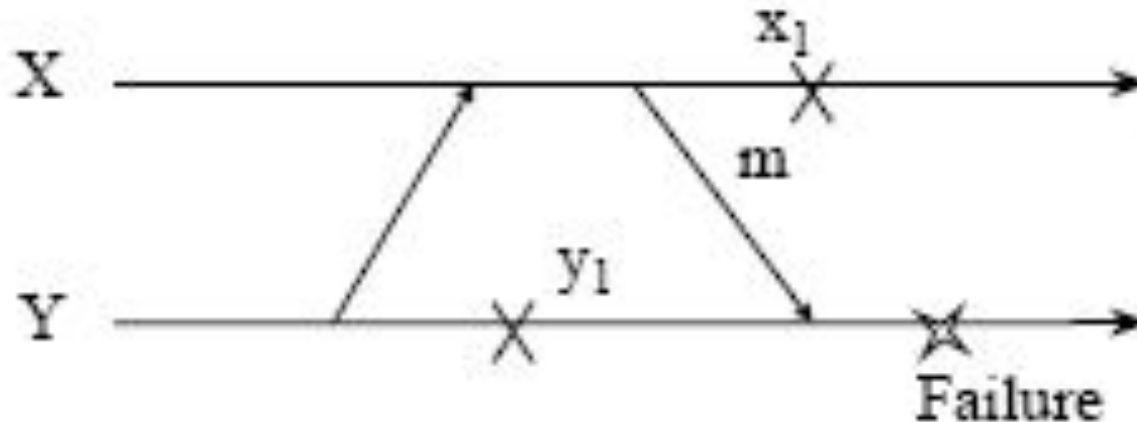
What's Wrong?



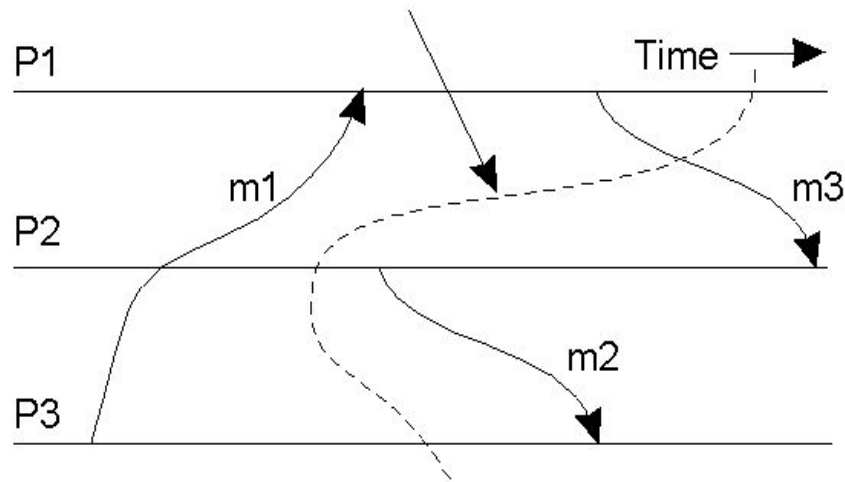
- Result:
 - Global state has record of the receive event but no send event **violating the happens-before concept!!**
- An orphan message is a message whose receiving event is recorded in the checkpoint, but its sending event is lost.

Lost Messages

- A message whose sending event is recorded, but its receiving event is not recorded.

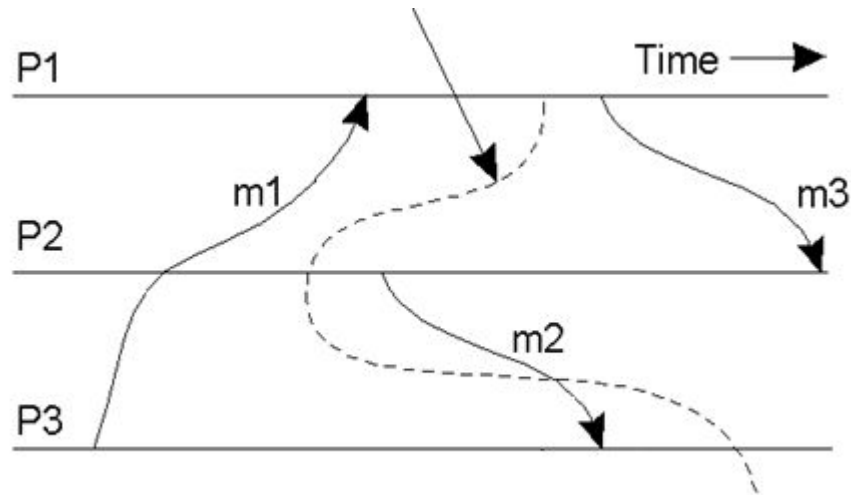


Cut



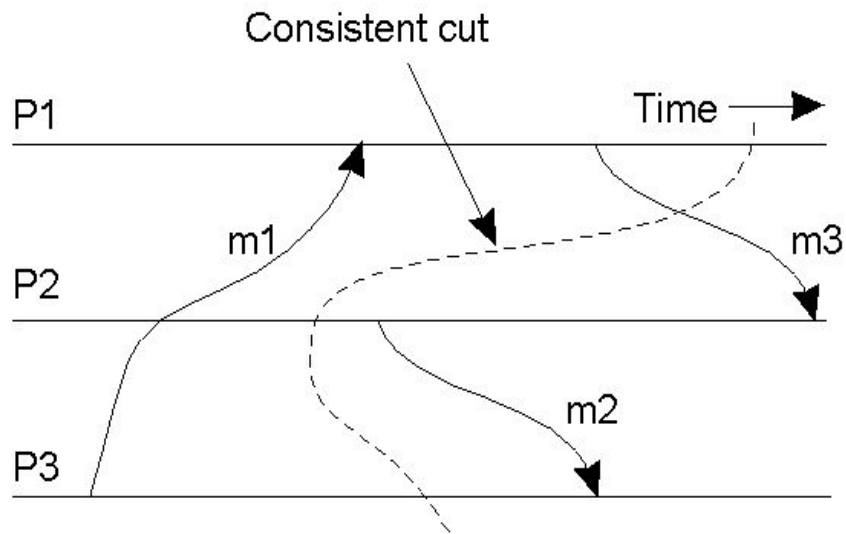
A consistent cut (**meaningful global state**) ?

Cut

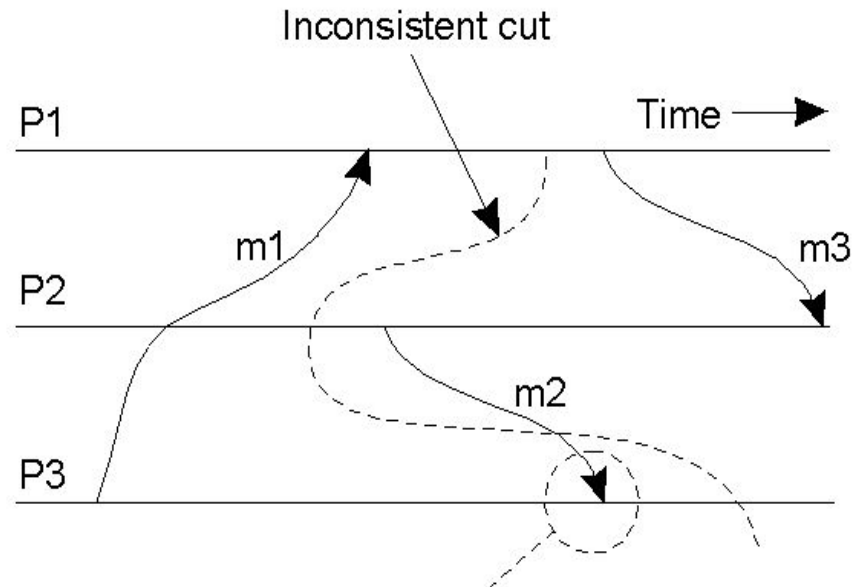


A consistent cut (**meaningful global state**) ?

Cuts



(a)



Sender of m2 cannot
be identified with this cut

(b)

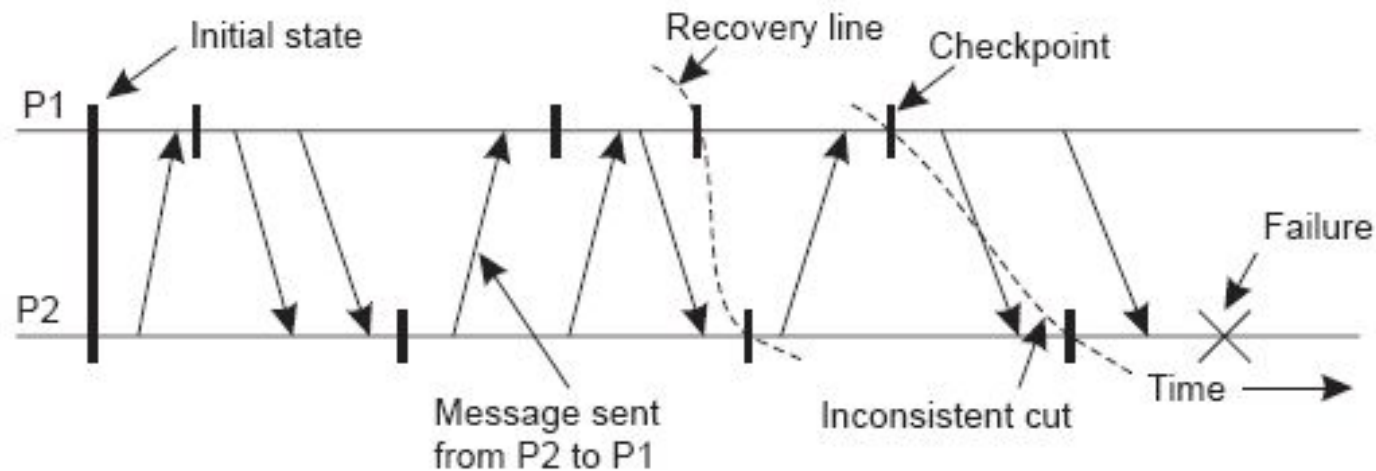
- a) A consistent cut (**meaningful global state**)
- b) An inconsistent cut

Consistent Checkpoints

- A process saves its local state on the stable storage, which is called a local checkpoint.
- The process of saving local states is called local checkpointing.
- All the local checkpoints, one from each site, collectively form a global checkpoint.
- A global checkpoint is a **strongly consistent** set of checkpoints if there is no orphan and no lost message.
- A global checkpoint is a **consistent** set of checkpoints if there is no orphan message.

Checkpointing

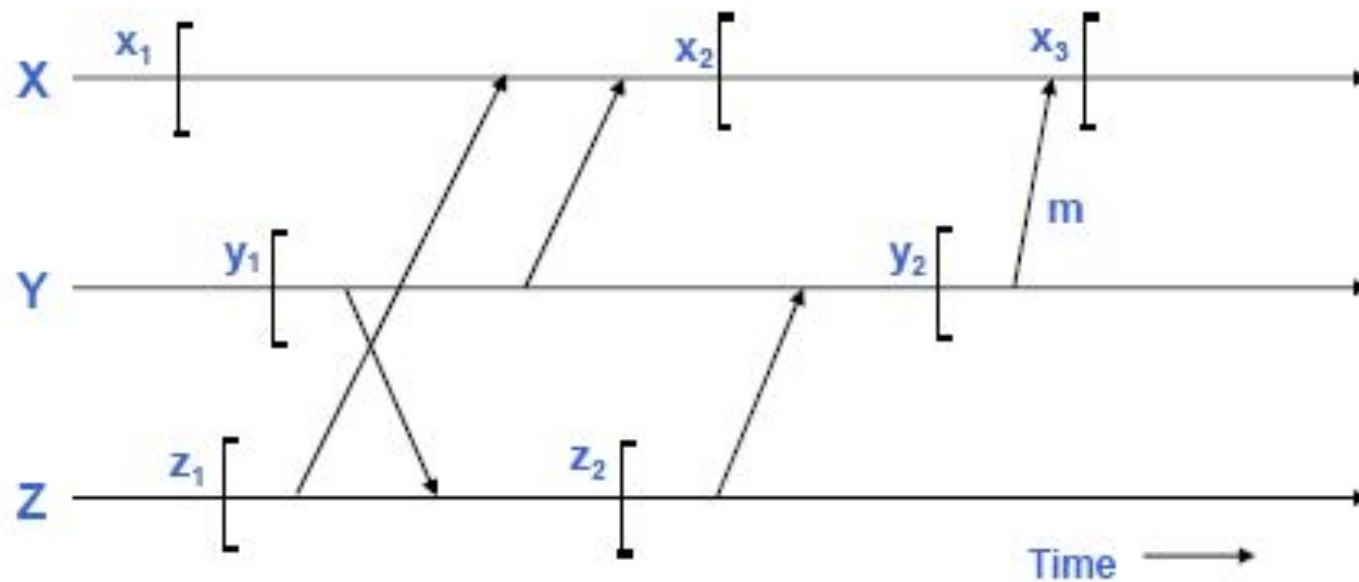
- The most recent distributed snapshot in a system is also called the *recovery line*



Independent Checkpointing

- Each processes periodically checkpoints independently of other processes
- Upon a failure, work backwards to locate a consistent cut
- Problem: if most recent checkpoints form inconsistent cut, will need to keep rolling back until a consistent cut is found
- Cascading rollbacks can lead to a domino effect.

Domino Effect:



- Rollback of X does not affect other processes.
- Rollback of Z requires all three processes to roll back to their very first recovery points.

- Domino Effect:

The domino effect is a chain reaction that occurs when a small change causes a similar change nearby, which then will cause another similar change, and so on in linear sequence. The term is best known and is used as an analogy to a falling row of dominoes.



Coordinated Checkpointing

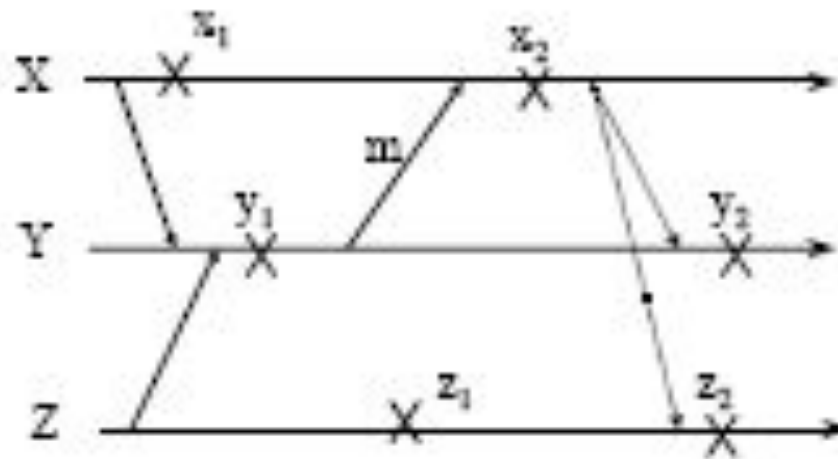
- To solve this problem, systems can implement *coordinated checkpointing*
- Take a distributed snapshot
- Upon a failure, roll back to the latest snapshot
 - All process restart from the latest snapshot
- Algorithm for distributed global snapshots is not particularly efficient for large systems
- Another way to do it is to use a two-phase blocking protocol (with some coordinator) to get every process to checkpoint its local state “simultaneously”

Coordinated Checkpointing

- Make sure that processes are synchronized when doing the checkpoint
- Two-phase blocking protocol
 1. Coordinator multicasts *CHECKPOINT_REQUEST*
 2. Processes take local checkpoint
 - Delay further sends
 - Acknowledge to coordinator
 - Send state
 3. Coordinator multicasts *CHECKPOINT_DONE*

An Optimization

- Sometimes, it is not necessary to ask all processes to take checkpoints for each checkpointing initiation.



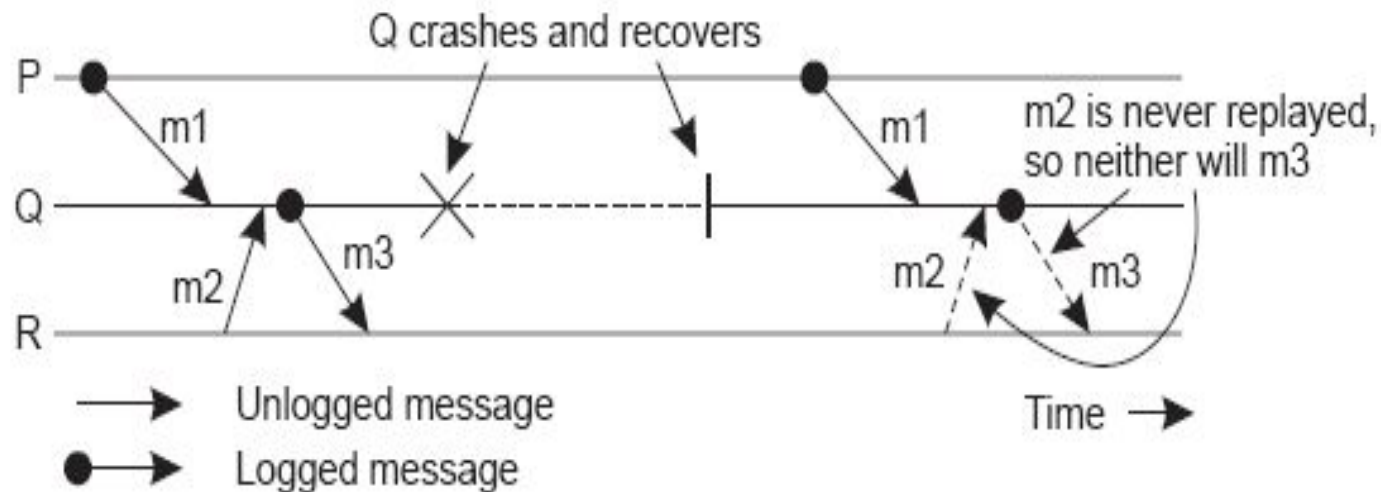
Message Logging

- Checkpointing is expensive
 - All processes restart from previous consistent cut
 - Taking a snapshot is expensive
 - Infrequent snapshots $\Rightarrow$ all computations after previous snapshot will need to be redone [wasteful]
- Combine checkpointing (expensive) with message logging (cheap)
 - Take infrequent checkpoints
 - Log all messages between checkpoints to local stable storage
 - To recover: simply replay messages from previous checkpoint
 - Avoids recomputations from previous checkpoint

Message Logging

- Message-logging schemes can be characterized as *pessimistic* or *optimistic*
- **Pessimistic message logging**: an incoming message is logged before it is processed.
 - Too much overhead, and
 - slows down computation
- **Optimistic message logging** : processes continue to perform the computation, and store messages in memory, which will be logged at certain intervals.
 - More rollbacks during failure,
 - may still have domino effects.

Message Logging



- An example of an incorrect replay of messages